Running GRPO on Google Colab

Reinforcement learning for the pure-C engine

Upload - Generate - Reinforce - Measure - Promote

GRPO is the most memory-hungry thing this engine does. It holds a 621 MB checkpoint, samples a group of rollouts from it, keeps every rollout's log-probabilities, and then runs a forward and backward pass per rollout. On a desktop with other work in flight that is enough to exhaust system memory and have the run killed mid-flight - which is exactly what happened on 5 September 2026, at step 14 of a 20-step evaluation.

Colab fixes that. A standard runtime gives around 12.7 GB of RAM and roughly 100 GB of disk, so the checkpoints and their throwaway copies stop competing with your desktop. This guide covers the whole loop end to end: what to upload, how to regenerate the task data there, how to run GRPO, how to read the numbers it prints, and how to decide whether a result is worth promoting.

Read this first

Colab buys you MEMORY, not necessarily SPEED. Section 1.3 explains why the GPU does not accelerate the rollout phase and why a free-tier runtime can be slower per step than a 3-thread desktop run. Read 1.3 before assuming a move to Colab makes the run faster.

Version: engine @ agent-1 (GRPO + OpenCL GPU backend)
Generated: 5 September 2026
Generated by tools/guidepdf (Go + gofpdf). Regenerate with: cd tools/guidepdf && go run . -doc colab

Contents

1 Why move GRPO to Colab 4
- 1.1 What actually went wrong locally 4
- 1.2 What Colab fixes, and what it does not 4
- 1.3 Why the GPU does not rescue the rollout phase 4

2 What has to go up 5
- 2.1 The manifest 5
- 2.2 Use Drive for the checkpoint, not the upload widget 5

3 Setting up the runtime 6
- 3.1 Pick the runtime 6
- 3.2 Get the binary 6
- 3.3 Install the OpenCL loader 6
- 3.4 Validate before you train 6

4 Regenerating the task data 7
- 4.1 The two-step pipeline 7
- 4.2 Why data/wiki matters more than it looks 7
- 4.3 A held-out set for honest evaluation 7

5 Running GRPO 8
- 5.1 The command 8
- 5.2 The flags that matter 8
- 5.3 Run it detached, and keep the log 8

6 Reading the output 9
- 6.1 Flat groups are the number to watch 9
- 6.2 Reference numbers from 5 September 2026 9
- 6.3 The reward breakdown 9

7 The hyperparameters, and what is still unknown 10
- 7.1 What was measured 10
- 7.2 The untested lever 10
- 7.3 How to tell quickly whether it is working 10

8 Memory and disk discipline 11
- 8.1 Every run writes a full checkpoint 11
- 8.2 Evaluating without training 11

9 Deciding whether to promote 12
- 9.1 The gate 12
- 9.2 What promotion means 12

10 Troubleshooting 13

11 Appendix: the whole notebook 14
- 11.1 Setup 14
- 11.2 Assets 14
- 11.3 Validate 14
- 11.4 Data 14
- 11.5 Baseline, train, compare 14
- 11.6 Gate and keep 15

1 Why move GRPO to Colab

Because GRPO is the one workload here that runs out of memory, and because a killed run wastes an hour rather than a minute.

1.1 What actually went wrong locally

A paired before/after evaluation was run on 5 September 2026: two GRPO invocations in sequence, each loading a 621 MB checkpoint and each writing another 621 MB checkpoint at the end. The first completed. The second was killed by the operating system at step 14 of 20 with a low-memory condition. Nothing was corrupted and the partial result was still usable, but an hour of compute produced three quarters of an answer.

The arithmetic is unforgiving. One resident model is 621 MB. Each finished run writes a fresh 621 MB checkpoint to disk. Run three experiments back to back without cleaning up and roughly 1.9 GB of dead weight is sitting in the working directory while the next model loads on top of it.

1.2 What Colab fixes, and what it does not

Constraint	Desktop	Colab (standard)
`System RAM`	shared with everything else	~12.7 GB dedicated
`Disk for checkpoints`	your own drive	~100 GB ephemeral
`CPU threads`	set TF_THREADS=3 to stay usable	~2 vCPU on the free tier
`GPU`	integrated GPU is SLOWER than the CPU	T4 available, but see 1.3
`Interruption risk`	killed by memory pressure	disconnects when idle

> WARNING: Colab is a memory fix, not a speed fix. On the free tier you get about two vCPUs, which is FEWER than the three threads a local run is limited to. Per-step wall clock can be worse.

1.3 Why the GPU does not rescue the rollout phase

The OpenCL backend offloads a matmul only when the M dimension is large enough to pay for the transfer, governed by TF_GPU_MIN_M. Single-token decode has M=1: it is memory-bandwidth bound, so it deliberately stays on the CPU. GRPO spends most of its time sampling rollouts, and sampling is single-token decode.

So the GPU helps the policy-gradient forward and backward passes, which run over the full sequence, and does nothing for the sampling that dominates the clock. Enable it, but do not budget for a large speedup.

> NOTE: Measured locally at TF_THREADS=3: about 84 seconds per step on the hard task file, with a range of 19 to 154 seconds depending on prompt length and how many rollouts hit the 96-token cap.

2 What has to go up

Most of what you need is already in the repository. The two big pieces are not, because .gitignore excludes generated weights and datasets - the generators are the tracked artifact, not their output.

2.1 The manifest

File	Size	In git?	How to get it there
`build/transformer-linux-x86_64-gpu`	1.1 MB	yes	clone the repo, or upload directly
`model/smollm/reason_e2.bin`	621 MB	no	Google Drive - do not use files.upload()
`model/smollm/tok.hf`	421 KB	no	Drive, or re-export with tools/export_tokenizer.py
`data/wiki/ (383 files)`	7.4 MB	no	zip it and upload; see 4.2 for why it matters
`data/so_commands.txt`	8.6 KB	yes	comes with the clone
`tools/hard_tasks.py`	-	yes	comes with the clone
`tools/grpo_tasks.py`	-	yes	comes with the clone

2.2 Use Drive for the checkpoint, not the upload widget

files.upload() pushes through the browser and is slow and failure-prone at 621 MB. Mount Drive instead: the checkpoint is copied once, then survives every runtime restart, which matters because Colab will disconnect an idle session and you do not want to re-upload 621 MB each time.

```
from google.colab import drive
drive.mount('/content/drive')

# one-time: put reason_e2.bin and tok.hf in a Drive folder, then
!mkdir -p model/smollm
!cp '/content/drive/MyDrive/llm-project/reason_e2.bin' model/smollm/
!cp '/content/drive/MyDrive/llm-project/tok.hf'        model/smollm/
```

> TIP: Copy the checkpoint OUT of Drive onto the local disk before training. Reading a 621 MB file repeatedly across the Drive FUSE mount is far slower than local disk.

3 Setting up the runtime

3.1 Pick the runtime

Runtime, then Change runtime type. A GPU runtime is worth taking even given section 1.3, because the update passes do benefit and the GPU runtimes tend to come with more RAM. If you have Colab Pro, a high-RAM runtime removes the memory question entirely.

3.2 Get the binary

The repository ships a cross-compiled Linux binary, so nothing has to be built on Colab:

```
!git clone <your-remote> llm-project
%cd llm-project
!chmod +x build/transformer-linux-x86_64-gpu
!./build/transformer-linux-x86_64-gpu gpu-info
```

If you build it yourself instead, produce it on your own machine with either of these, and upload the result - Colab is not set up to cross-compile this tree:

```
powershell -File tools\build_gpu.ps1 -Only colab   # Windows
make cross-gpu                                      # Linux/macOS
```

3.3 Install the OpenCL loader

Colab ships the NVIDIA driver, which provides OpenCL, but usually not the vendor-neutral loader or the ICD registration file that points the loader at NVIDIA. Both are one line. Skipping this is survivable - gpu.c falls back to opening libnvidia-opencl.so.1 directly - but installing the loader is the supported path.

```
!apt-get -qq update && apt-get -qq install -y ocl-icd-libopencl1 clinfo > /dev/null
!mkdir -p /etc/OpenCL/vendors && echo libnvidia-opencl.so.1 > /etc/OpenCL/vendors/nvidia.icd
!clinfo -l
```

3.4 Validate before you train

Run the smoke test first. Part [30] is the one that matters: it runs the forward matmul and both backward matmuls on deliberately ragged shapes - not multiples of the kernel's 16-wide tile or 64-column block - and requires the GPU to agree with the CPU. It also checks that the backward kernels accumulate into pre-seeded gradients instead of overwriting them.

```
!./build/transformer-linux-x86_64-gpu 2>&1 | tail -n 30
```

> WARNING: A fast wrong answer is not a speedup. If part [30] is not green, stop - every number produced after that point is meaningless.

> NOTE: tools/colab_gpu_test.ipynb automates this entire section and adds a CPU-versus-GPU benchmark. Run it once before the first GRPO job.

4 Regenerating the task data

Task files are gitignored, so they are generated on Colab rather than uploaded. This is deterministic given --seed, so the file you build there is the file you would have built at home.

4.1 The two-step pipeline

Generate, then gate. The gate is not optional: an ungrounded reference would make the reward pay out for a hallucination, and reward and truth would come apart.

```
!python tools/hard_tasks.py --out data/hard_raw.txt --n 600 \
    --wiki data/wiki --commands data/so_commands.txt --reason
!python tools/grpo_tasks.py --out data/grpo_hard.txt data/hard_raw.txt
```

Expect all 600 to survive the gate. If a run reports drops, a template in hard_tasks.py has produced a reference that is not grounded in its own prompt - fix the template, do not loosen the gate.

4.2 Why data/wiki matters more than it looks

slugs_from_wiki falls back to six generic slugs when the directory is missing, and the write-content pool is derived from those same slugs. With the fallback, the content vocabulary collapses to roughly ten words - small enough for the model to memorize the content set instead of learning to copy content out of the prompt. That failure has been observed in this project before: novel first-write content copied fine while the second write's body was dropped to empty.

> WARNING: Upload data/wiki. Without it the generator still runs and still passes the gate, but the tasks quietly get easier and the flat-group rate climbs back up.

```
# on your machine: Compress-Archive data/wiki wiki.zip
!unzip -q wiki.zip -d data/
!ls data/wiki | wc -l      # expect 383
```

4.3 A held-out set for honest evaluation

Use a different generator seed. This gives tasks from the same distribution with zero line overlap, which is what makes a before/after comparison meaningful.

```
!python tools/hard_tasks.py --out data/hard_eval_raw.txt --n 200 \
    --wiki data/wiki --commands data/so_commands.txt --reason --seed 4242
!python tools/grpo_tasks.py --out data/grpo_hard_eval.txt data/hard_eval_raw.txt
```

5 Running GRPO

5.1 The command

```
!./build/transformer-linux-x86_64-gpu grpo data/grpo_hard.txt \
    --model model/smollm/reason_e2.bin \
    --hf-tok model/smollm/tok.hf \
    --out model/smollm/grpo_hard.bin \
    --steps 100 --groups 4 --lr 1e-4 --verbose
```

5.2 The flags that matter

Flag	Default	What it does
`--group`	8	rollouts sampled per task; the group's own mean is the baseline that replaces a value network
`--groups`	1	task groups accumulated into one optimizer step; this is the batch size, and 1 is a very noisy gradient
`--steps`	100	optimizer steps
`--lr`	1e-5	learning rate; RL wants far less than SFT, but see section 7
`--temp`	1.0	sampling temperature - the exploration knob
`--lora-r`	8	LoRA rank, base weights frozen
`--clip-eps`	0.2	PPO-style ratio clip
`--kl`	0 (off)	KL penalty toward the frozen base; costs one extra forward per rollout
`--seed`	-	seeds BOTH task selection and token sampling

5.3 Run it detached, and keep the log

A Colab cell that runs for an hour is a cell you cannot look at while it works. Redirect to a log and tail it from another cell.

```
import subprocess
cmd = ['./build/transformer-linux-x86_64-gpu', 'grpo', 'data/grpo_hard.txt',
       '--model', 'model/smollm/reason_e2.bin', '--hf-tok', 'model/smollm/tok.hf',
       '--out', 'model/smollm/grpo_hard.bin', '--steps', '100',
       '--groups', '4', '--lr', '1e-4', '--verbose']
log = open('grpo_run.log', 'w')
proc = subprocess.Popen(cmd, stdout=log, stderr=subprocess.STDOUT)
print('pid', proc.pid)

# then, in a separate cell, as often as you like:
!tail -n 20 grpo_run.log
```

> TIP: Colab disconnects idle sessions. A browser tab left open on the notebook counts as idle if nothing renders - tailing the log periodically also keeps the session alive.

6 Reading the output

A GRPO step line looks like this:

```
    step    16/100  reward 0.9375  (running 0.6864, 4 flat groups skipped)  102.0s
```

6.1 Flat groups are the number to watch

If every rollout in a group scores identically, every advantage is zero and the group is skipped before any forward pass. GRPO learns from variance: where the model is uniformly right or uniformly wrong there is no signal at all. A run reporting mostly flat groups is not training, however healthy its reward looks.

This is not hypothetical. Running GRPO against the old SFT-derived task file produced five flat groups out of five at a mean reward of exactly 1.0000 - a perfect score and a completely useless run. The fix was harder tasks, not a hotter temperature.

6.2 Reference numbers from 5 September 2026

Task file	Temp	Flat groups	Mean reward
`data/grpo_tasks.txt (old, SFT-derived)`	1.0	5 of 5	1.0000
`data/grpo_tasks.txt`	1.5	2 of 5	0.9475
`data/grpo_hard.txt`	1.0	2 of 10	0.6712
`data/grpo_hard_eval.txt (held out)`	1.0	7 of 20	0.6144

> NOTE: A mean reward near 0.6 to 0.7 with most groups live is the target zone: real headroom above, real spread below. A mean near 1.0 means the tasks are too easy to teach anything.

6.3 The reward breakdown

With --verbose each group prints one scored sample. The three terms are independently verifiable against something outside the model - the prompt text or the reference action - and the breakdown tells you WHY reward moved:

Term	Weight	Meaning
`format`	0.20	exactly one well-formed, properly closed tool or done block, nothing trailing
`grounded`	0.30	every argument occurs in the prompt as a whole token - catches hallucinated filenames
`match`	0.50	the parsed action equals the reference; partial credit for right tool, wrong argument

Reward hacking shows up here first. A total that climbs while match stays flat means the model has found a cheap way to satisfy format and grounding without doing the task.

> WARNING: A total that climbs while match stays flat is not progress. Check the breakdown, not just the total.

7 The hyperparameters, and what is still unknown

As of 5 September 2026 the task data is solved and the optimization is not. This section is the open question, stated honestly rather than dressed up as a recipe.

7.1 What was measured

A 20-step run at the defaults - lr 1e-5, one group per step, LoRA rank 8 - changed the weights but produced no behavioral change whatsoever. Two independent checks agree:

- The first 10 steps of the 20-step run were bit-identical to a separate 10-step run: same rewards, same running means, same flat-group counts, across 80 rollouts.
- On 15 paired held-out steps - 120 rollouts, zero overlap with anything trained on - the before and after checkpoints produced identical output at every step.
- The checkpoints differ by SHA-256, so gradient is flowing and the plumbing is sound. The updates are simply too small to change a single sampled token.

7.2 The untested lever

Eight rollouts per update is a very noisy gradient, and 1e-5 is a small step for a LoRA adapter on a 135M model. The obvious next experiment raises both:

```
--groups 4 --lr 1e-4      # 32 rollouts per update, 10x the step size
```

That is roughly four times the compute per step, which is precisely why it belongs on Colab rather than on a desktop being used for anything else.

> WARNING: This has NOT been run. It is the reasoned next experiment, not a validated recipe. Treat the first result as information, not as a checkpoint to promote.

7.3 How to tell quickly whether it is working

Do not wait for a hundred steps. Run twenty, then compare against the same twenty steps at the old settings. Identical rewards step for step means the policy has not moved and the settings are still too weak - that comparison costs twenty minutes and saves two hours.

8 Memory and disk discipline

This is the section that exists because a run was killed. Colab gives you room, not immunity.

8.1 Every run writes a full checkpoint

There is deliberately no mid-run checkpointing in GRPO: saving would mean merging the adapters into the base, and the merge resets B to zero, after which the adapter gradient is identically zero and training stalls. A run therefore saves exactly once, at the end - and that save is a full 621 MB, not a small adapter file.

- Delete throwaway checkpoints as soon as you have read the number you wanted from them.
- Write experimental output to /tmp so a runtime restart clears it automatically.
- Copy only checkpoints you actually intend to keep back to Drive.

```
!ls -la --block-size=M model/smollm/*.bin /tmp/*.bin 2>/dev/null
!rm -f /tmp/probe.bin /tmp/eval_*.bin
!df -h /content | tail -1
!free -g | head -2
```

8.2 Evaluating without training

Passing --lr 0 turns the trainer into a scorer: it still samples the full group per task and reports the reward, but no parameter changes. This was verified by hashing - a run at --lr 0 writes a checkpoint byte-identical to its input.

```
# paired evaluation: identical seed and task file, only --model differs
!./build/transformer-linux-x86_64-gpu grpo data/grpo_hard_eval.txt \
    --model model/smollm/reason_e2.bin --hf-tok model/smollm/tok.hf \
    --out /tmp/eval_before.bin --steps 20 --lr 0 --seed 7 --verbose
```

> NOTE: Both task selection and token sampling are seeded from --seed, so the same seed on the same task file draws the same tasks in the same order. That is what makes the comparison paired rather than two independent samples.

9 Deciding whether to promote

A finished run is not a better model. The project rule is that nothing is promoted without measurement, and the standing risk in this tree has been features built but never validated end to end.

9.1 The gate

```
!python tools/remeasure.py model/smollm/grpo_hard.bin
!python tools/probe3.py    model/smollm/grpo_hard.bin
```

remeasure.py runs the full set - probe3, sft_memtest and eval_critic - against the recorded baseline, and prints a question mark for anything it could not parse, so a missing number never reads as a pass. It was built precisely so that a probe cannot be skipped.

> WARNING: sft_memtest execs the engine argv verbatim, so it needs an ABSOLUTE path to the binary. probe3 and eval_critic resolve paths internally and once hid this problem for a whole run.

9.2 What promotion means

- Copy the checkpoint back to Drive before the runtime disconnects and takes it with it.
- Record the numbers in notes.md and memory.md, including the ones that did not improve.
- If nothing improved, that is a result worth committing too - it stops the next session from re-running the same experiment.

10 Troubleshooting

Symptom	Cause and fix
`Every group reported flat`	The tasks are too easy for this checkpoint. Rebuild with tools/hard_tasks.py; do not just raise --temp, which only adds downward noise if the model is already at the ceiling.
`Mean reward is exactly 1.0000`	Same cause. The model solves every sampled task; there is no gradient anywhere in the run.
`Rewards identical across two runs`	The policy is not moving. Raise --lr and --groups; see section 7.
`gpu-info says NO USABLE DEVICE`	A correct and supported state - every matmul runs on the CPU. Check the runtime is a GPU runtime and that section 3.3 ran.
`Selftest part [30] fails`	Stop. The GPU kernels disagree with the CPU; every later number is meaningless.
`Process killed, no error`	Out of memory. Delete stale .bin files and restart the runtime; see section 8.
`Session disconnected mid-run`	Colab idle timeout. The run is gone and no checkpoint was written - GRPO saves only at the end. Tail the log periodically.
`Tasks dropped by grpo_tasks.py`	A generated reference is not grounded in its own prompt. Fix the template in hard_tasks.py; never loosen the gate.
`No speedup from --gpu`	Expected. Rollout decode is M=1 and stays on the CPU by design; see 1.3.

11 Appendix: the whole notebook

Every cell in order, assuming a GPU runtime and the checkpoint already in Drive.

11.1 Setup

```
!nvidia-smi
!apt-get -qq update && apt-get -qq install -y ocl-icd-libopencl1 clinfo > /dev/null
!mkdir -p /etc/OpenCL/vendors && echo libnvidia-opencl.so.1 > /etc/OpenCL/vendors/nvidia.icd

from google.colab import drive
drive.mount('/content/drive')

!git clone <your-remote> llm-project
%cd llm-project
!chmod +x build/transformer-linux-x86_64-gpu
```

11.2 Assets

```
!mkdir -p model/smollm
!cp '/content/drive/MyDrive/llm-project/reason_e2.bin' model/smollm/
!cp '/content/drive/MyDrive/llm-project/tok.hf'        model/smollm/
!unzip -q '/content/drive/MyDrive/llm-project/wiki.zip' -d data/
!ls data/wiki | wc -l
```

11.3 Validate

```
!./build/transformer-linux-x86_64-gpu gpu-info
!./build/transformer-linux-x86_64-gpu 2>&1 | tail -n 30   # part [30] must be green
```

11.4 Data

```
!python tools/hard_tasks.py --out data/hard_raw.txt --n 600 \
    --wiki data/wiki --commands data/so_commands.txt --reason
!python tools/grpo_tasks.py --out data/grpo_hard.txt data/hard_raw.txt

!python tools/hard_tasks.py --out data/hard_eval_raw.txt --n 200 \
    --wiki data/wiki --commands data/so_commands.txt --reason --seed 4242
!python tools/grpo_tasks.py --out data/grpo_hard_eval.txt data/hard_eval_raw.txt
```

11.5 Baseline, train, compare

```
BIN = './build/transformer-linux-x86_64-gpu'
M   = 'model/smollm/reason_e2.bin'
TOK = 'model/smollm/tok.hf'

# 1. baseline on held-out tasks (no training: --lr 0)
!$BIN grpo data/grpo_hard_eval.txt --model $M --hf-tok $TOK \
    --out /tmp/eval_before.bin --steps 20 --lr 0 --seed 7 --verbose

# 2. the actual run
!$BIN grpo data/grpo_hard.txt --model $M --hf-tok $TOK \
    --out model/smollm/grpo_hard.bin --steps 100 --groups 4 --lr 1e-4 --verbose

# 3. the same held-out tasks, same seed, new model
!$BIN grpo data/grpo_hard_eval.txt --model model/smollm/grpo_hard.bin \
    --hf-tok $TOK --out /tmp/eval_after.bin --steps 20 --lr 0 --seed 7 --verbose

!rm -f /tmp/eval_before.bin /tmp/eval_after.bin
```

11.6 Gate and keep

```
!python tools/remeasure.py model/smollm/grpo_hard.bin
!cp model/smollm/grpo_hard.bin '/content/drive/MyDrive/llm-project/'
```

> NOTE: If step 3 prints the same rewards as step 1 at every step, the run changed nothing. That is a real result: record it, and raise --lr or --groups before spending another hour.